Introduction to Machine Learning

Machine Learning with Python — Lecture 1, Session 1 (Oct 19)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

- **What is machine learning?** — the paradigm shift from rules to data
- **Types of machine learning** — supervised, unsupervised, semi- and self-supervised, reinforcement
- **The ML workflow** — and where the real work happens
- **Over- and underfitting** — the central tension of the field

Session 1 of 4 today. Please ask questions at any point.

Practical information

- Slides are Marimo notebooks, exercises are Jupyter notebooks — all in this repository
- Clone the repo and follow the setup in the `README.md` (`uv sync`, then `uv run jupyter lab`)
- Today's exercises: `notebooks/01/01_introduction_{beginner,intermediate,advanced}.i`
- All exercises use `numpy`, `matplotlib` and `scikit-learn` — nothing else to instal

What is machine learning?

> *"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E." — Mitchell, 1997*

In one sentence:

Machine learning is the discipline of building systems that learn rules from data, instead of being explicitly programmed.

- Traditional programming: *data + program → output*
- Machine learning: *data + output → program* — the learned **model** is then applied to new, unseen data.

Why machine learning?

- Problems that are **hard to specify** but easy to demonstrate: spam detection, image recognition, machine translation
- Problems that **adapt over time**: recommender systems, fraud detection
- Problems at **scale**, where hand-written rules break down

ML is not magic — it is a tool that shines when the data is right and the question is well-posed.

Types of machine learning

Type	What the algorithm sees	Goal
Supervised	inputs **+ labels**	predict the label of new inputs
Unsupervised	inputs only	discover structure (clusters, low-dimensional representations)
Semi-supervised	few labels + many unlabeled inputs	leverage both
Self-supervised	inputs only	create labels from the data itself (denoising, masked words)
Reinforcement	interactions + rewards	learn actions that maximise reward

This course is mostly about **supervised learning**.

Supervised learning

We are given a dataset of (input, target) pairs (x_i, y_i) — the **examples** — and we want a function that turns a new input into a good prediction:

$$f(x) \approx y$$

In plain words: look at many examples, guess the rule that produced them, and apply it to inputs you have never seen.

- **Classification** — y is a discrete label (spam / not spam, healthy / diseased)
- **Regression** — y is a continuous number (price, temperature)

Algorithms in this course: **linear regression** (session 2), **decision trees** (session 3), **random forests** (session 4), and **neural networks** on day 2 (opening with **logistic regression**).

Unsupervised, semi-supervised, self-supervised

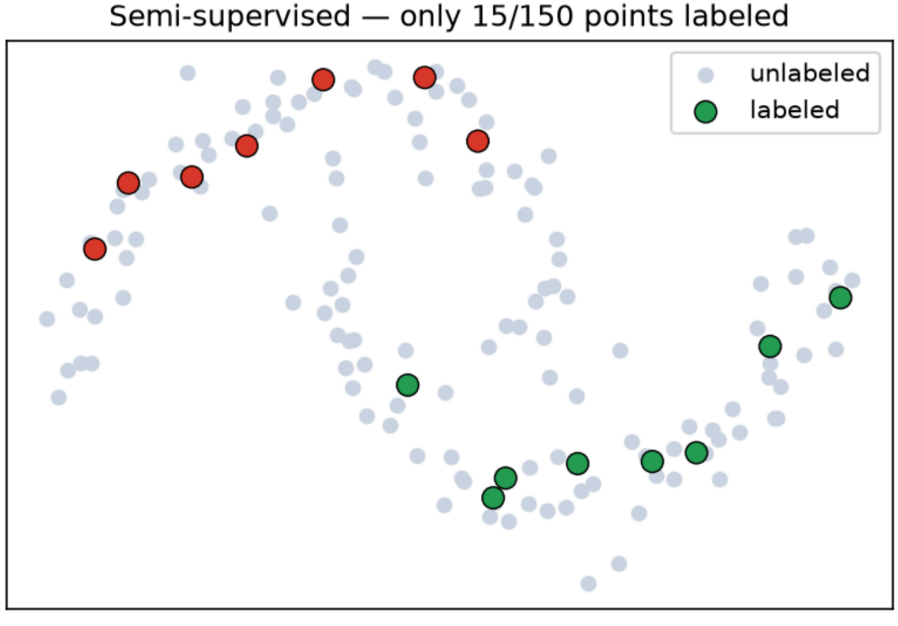
- **Unsupervised:** only inputs. *Clustering* — group similar points; *dimensionality reduction* — find a compact representation.
- **Semi-supervised:** a few labeled + many unlabeled points (labeling is often expensive — think medical images). The unlabeled points reveal the *shape* of the data, which constrains the decision boundary.
- **Self-supervised:** the data creates its own supervision — predict a masked word, reconstruct a denoised image. The engine behind modern LLMs.
- **Reinforcement learning:** an agent acts, the environment rewards. Out of scope for this course.

One dataset, three paradigms

The slider controls **how many points are labeled**:

- 100 % labeled → **supervised** classification
- a few labeled → **semi-supervised** learning
- none labeled → **unsupervised** learning (e.g. clustering)

Fraction of labeled points 0.1



The ML workflow

1. **Frame the problem** — what is the task, what counts as success?
2. **Collect & explore the data** — plots, summary statistics
3. **Preprocess** — missing values, encoding, scaling
4. **Split** — training / validation / test sets
5. **Choose & train a model** — fit to the training set
6. **Evaluate** — on held-out data, never on the training set
7. **Iterate**

Overfitting and underfitting

- **Underfitting:** the model is too simple — it misses the pattern (high *training* error).
- **Overfitting:** the model memorises noise — great on training data, poor on new data.

The cure: more data, simpler models, **regularisation** — and always keeping a **held-out test set** to detect it.

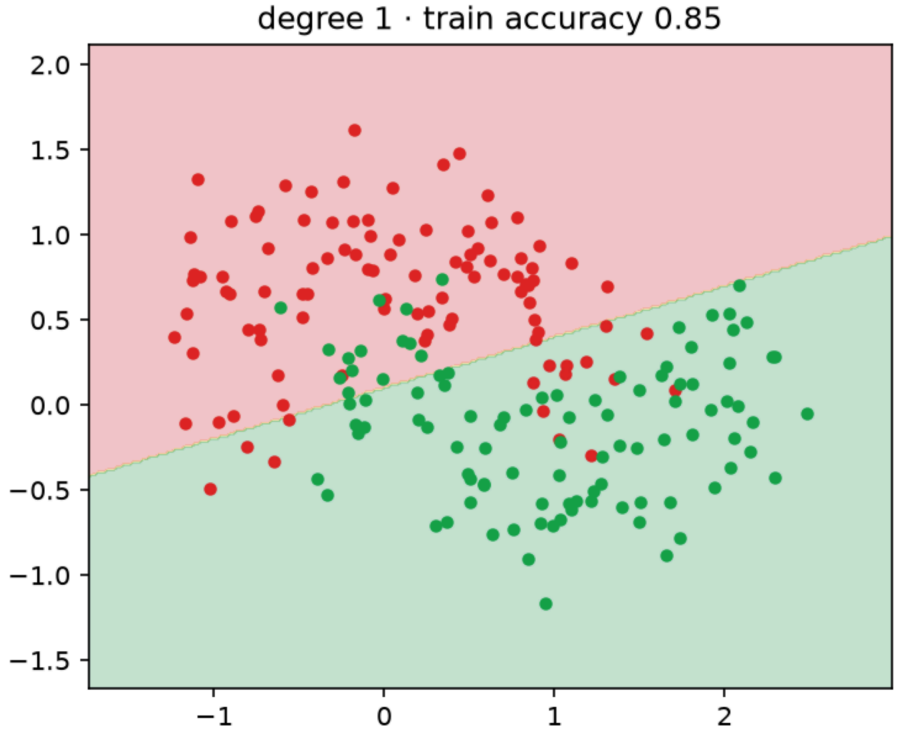
Below: logistic regression on two moons with polynomial features of increasing degree — watch the boundary wiggle.

Interactive demo — model complexity

Slide the **degree** from 1 to 9 to change how flexible the model is.

- degree 1 → a straight line: **underfitting**
- a few degrees → a smooth curve that follows the shape well
- degree 9 → a wiggly boundary: **overfitting**

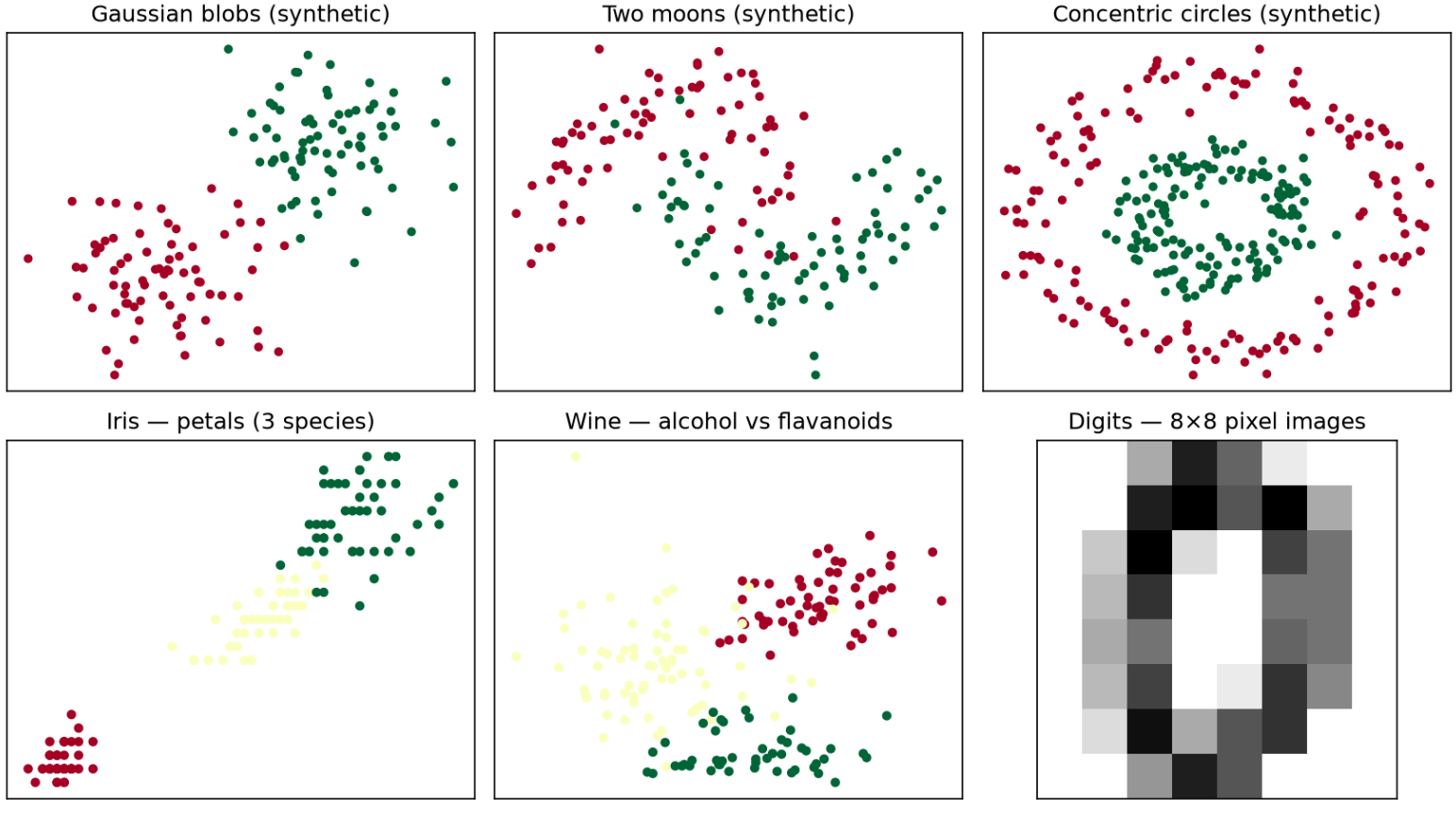
polynomial degree 1



The datasets of this course

Everything runs offline — the datasets ship with scikit-learn.

- **Synthetic** — `make_blobs`, `make_moons`, `make_circles` for clean illustrations
- **Iris** — 150 flowers, 4 features, 3 species
- **Wine** — 178 wines, 13 chemical features, 3 cultivars
- **Digits** — 1797 handwritten 8×8 images, 10 classes
- **Diabetes** — 442 patients, 10 features, a continuous target (regression)


Gaussian blobs (synthetic)
Two moons (synthetic)
Concentric circles (synthetic)
Iris — petals (3 species)
Wine — alcohol vs flavanoids
Digits — 8×8 pixel images

Summary

- **Machine learning** learns rules from data instead of being programmed.
- **Supervised** learning maps inputs to labels (classification) or numbers (regression); other paradigms (unsupervised, semi- and self-supervised, reinforcement) differ in what the algorithm sees.
- The **ML workflow** is mostly data work; the held-out **test set** is sacred.
- **Overfitting** (memorising) and **underfitting** (missing the pattern) are the two failure modes every model balances.

Next session: **linear regression**.